CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Computer Networks & Internet Protocols: Campus Master Edition (Prescribed Textbook)

Subject Track: Computer Networks (CN)

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Comprehensive campus placement textbook covering OSI 7-layer model, TCP/IP stack, CIDR subnetting, transport layer reliability, routing protocols (OSPF/BGP), application protocols (HTTP/2/3, DNS), network security, and socket programming.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Computer Networks (CN) on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: Network Architectures: OSI 7-Layer Model vs TCP/IP Stack

Ch 2: Physical & Data Link Layer Mechanics

Ch 3: Network Layer: IPv4, IPv6, Subnetting & CIDR

Ch 4: Routing Algorithms & Protocols: OSPF, BGP & Distance Vector

Ch 5: Transport Layer: TCP vs UDP In-Depth

Ch 6: Application Layer Protocols: HTTP, DNS, DHCP & TLS

Ch 7: Network Security, Firewalls & Cryptography

Ch 8: Solved Placement Networking Scenarios & System Diagnostics

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: Computer Networks (CN)

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

? EXAM GROUNDING POLICY: All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Comprehensive campus placement textbook covering OSI 7-layer model, TCP/IP stack, CIDR subnetting, transport layer reliability, routing protocols (OSPF/BGP), application protocols (HTTP/2/3, DNS), network security, and socket programming.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Network Architectures: OSI 7-Layer Model vs TCP/IP Stack](#chapter-1)
- [Chapter 2: Physical & Data Link Layer Mechanics](#chapter-2)
- [Chapter 3: Network Layer: IPv4, IPv6, Subnetting & CIDR](#chapter-3)
- [Chapter 4: Routing Algorithms & Protocols: OSPF, BGP & Distance Vector](#chapter-4)
- [Chapter 5: Transport Layer: TCP vs UDP In-Depth](#chapter-5)
- [Chapter 6: Application Layer Protocols: HTTP, DNS, DHCP & TLS](#chapter-6)
- [Chapter 7: Network Security, Firewalls & Cryptography](#chapter-7)
- [Chapter 8: Solved Placement Networking Scenarios & System Diagnostics](#chapter-8)

CHAPTER 1

Network Architectures: OSI 7-Layer Model vs TCP/IP Stack

Key Principle / Exam Takeaway: Master the 7 layers of OSI, 4 layers of TCP/IP, encapsulation/decapsulation headers, and protocol data units (PDU).

Core Curriculum Topics Covered in this Chapter:

- OSI 7-Layer reference model: Physical, Data Link, Network, Transport, Session, Presentation, Application
- TCP/IP 4-Layer architectural model: Network Interface, Internet, Transport, Application
- Data Encapsulation and Decapsulation: Bits, Frames, Packets, Segments, Messages
- Peer-to-peer vs Client-Server networking paradigms
- Connection-oriented vs Connectionless service models

· 1. Core Concepts & In-Depth Explanation

Computer networks connect distributed systems to exchange data. Two foundational reference models govern internet communications: 1. **OSI 7-Layer Reference Model:** Theoretical model standardizing protocol boundaries:

- Layer 7 (Application): User interface protocols (HTTP, DNS, SSH, SMTP).
- Layer 6 (Presentation): Syntax, encryption/decryption (TLS), and compression.
- Layer 5 (Session): Session checkpointing and synchronization.
- Layer 4 (Transport): End-to-end reliability and port multiplexing (TCP, UDP).
- Layer 3 (Network): Logical IP routing and forwarding across subnet boundaries.
- Layer 2 (Data Link): Physical MAC addressing and framing across local links.
- Layer 1 (Physical): Bitstream transmission over physical media (copper, fiber, radio).

2. **TCP/IP 4-Layer Model:** The pragmatic standard of the internet: combines Layers 5-7 into the Application Layer, retains Transport and Internet layers, and combines Layers 1-2 into the Network Access (Link) Layer.

Encapsulation: As data moves down the stack, each layer wraps the payload with a header: `Application (Message) -> Transport (Segment) -> Network (Packet) -> Data Link (Frame) -> Physical (Bits)`.

· 2. Key Formulas, Rules & Complexity Classes

1. **Protocol Data Unit (PDU) Mapping:** $$\text{Application (Message)} \to \text{Transport (Segment)} \to \text{Network (Packet)} \to \text{Data Link (Frame)}$$

· 3. Step-by-Step Solved Examples

Example 1.1: Step-by-Step Packet Encapsulation Trace

- **Difficulty / Target Companies:** `Easy | Amazon, TCS, Infosys`
- **Problem Statement:** Trace the header additions when an HTTP GET request is transmitted over an Ethernet network.

Step-by-Step Walkthrough & Logic:

- Step 1: Application Layer creates HTTP GET request string.
- Step 2: Transport Layer appends TCP header (Source Port: 54321, Dest Port: 80, Sequence Numbers).
- Step 3: Network Layer appends IP header (Source IP: 192.168.1.10, Dest IP: 93.184.216.34, TTL: 64).
- Step 4: Data Link Layer appends Ethernet Header (Source MAC, Dest MAC) and trailer CRC Checksum (FCS).
- Step 5: Physical Layer modulates the frame into electrical/optical signals.

Complete Solved Code / Calculation:

```
// Encapsulated Ethernet Frame Structure:
// [ Preamble ][ Eth Header (MAC) ][ IP Header ][ TCP Header ][ HTTP Payload ][ Frame Check Seq
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates how each layer operates independently, maintaining modular separation of concerns.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: Which layer of the OSI model is responsible for routing packets across multiple intermediate routers?

Options:

- A) Data Link Layer
- B) Network Layer
- C) Transport Layer
- D) Physical Layer

Answer: **B) Network Layer** **Detailed Solution & Reasoning:** The Network layer handles logical addressing (IPv4/IPv6) and path determination (routing).

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Cisco)

- **Interview Question:** _What is the difference between a Switch (Layer 2) and a Router (Layer 3)?_
- **Model High-Scoring Answer:** A Switch operates at Layer 2 (Data Link Layer). It inspects MAC addresses and uses a MAC address table to forward frames within the *same* local area network (LAN/broadcast domain). A Router operates at Layer 3 (Network Layer). It inspects IP addresses and uses a routing table to forward packets *between* different networks and subnets, interconnecting distinct broadcast domains.
- **Interviewer Follow-Up & Pro Tip:** ? Clearly mention MAC addresses vs IP addresses and broadcast domain boundaries.

CHAPTER 2

Physical & Data Link Layer Mechanics

Key Principle / Exam Takeaway: Master Framing, Flow Control (Stop-and-Wait, Go-Back-N, Selective Repeat), Error Detection (CRC, Parity), and CSMA/CD.

Core Curriculum Topics Covered in this Chapter:

- Framing techniques: Character count, byte stuffing, bit stuffing (HDLC)
- Error Detection: Parity check, Checksum, Cyclic Redundancy Check (CRC) polynomial division
- Flow Control: Stop-and-Wait vs Sliding Window (Go-Back-N vs Selective Repeat)
- Media Access Control: CSMA/CD (Ethernet) collision detection and exponential backoff
- Address Resolution Protocol (ARP): Mapping IP to MAC addresses and ARP cache poisoning

· 1. Core Concepts & In-Depth Explanation

The Data Link Layer (DLL) transforms unreliable raw bit streams into node-to-node frame transmissions across a local physical link:

- **Framing:** Delimiting frame boundaries using **Bit Stuffing** (inserting a '0' after five consecutive '1's to prevent user data from mimicking the flag sequence `01111110`).
- **Error Detection via CRC:** Uses polynomial modulo-2 arithmetic (XOR). The sender divides the data by a generator polynomial and appends the remainder (FCS). The receiver divides the received frame by the same generator; a non-zero remainder signals corruption.
- **Sliding Window Flow Control:**
- **Go-Back-N (GBN):** Receiver accepts frames strictly in-order; on error, sender retransmits the entire window of N frames starting from the corrupted frame.
- **Selective Repeat (SR):** Receiver buffers out-of-order frames; sender retransmits only the specific lost or damaged frame.

· 2. Key Formulas, Rules & Complexity Classes

1. **Sliding Window Optimal Window Size:** $$W_{\text{opt}} = 1 + 2a, \quad \text{where } a = \frac{\text{Propagation Delay } (T_p)}{\text{Transmission Delay } (T_t)}$$ 2. **Selective Repeat Maximum Window Size:** $$W_s \le 2^{m-1} \quad \text{(to prevent sequence number ambiguity)}$$

· 3. Step-by-Step Solved Examples

Example 2.1: CRC Modulo-2 Polynomial Remainder Calculation

- **Difficulty / Target Companies:** `Medium | Cisco, TCS Digital`
- **Problem Statement:** Given data message $D = 1010000$ and generator polynomial $G(x) = x^3 + x + 1$ (binary 1011). Calculate the transmitted CRC frame.

Step-by-Step Walkthrough & Logic:

- Step 1: Generator has 4 bits ($k = 3$ zeros appended): $D' = 1010000000$.

- Step 2: Perform modulo-2 division (XOR) of 1010000000 by 1011.
- Step 3: Remainder obtained $= 011$.
- Step 4: Transmitted frame $= D + \text{Remainder} = 1010000011$.

Complete Solved Code / Calculation:

```
// Transmitted Frame: 1010000 011
// Verification at receiver: 1010000011 divided by 1011 yields exactly 0 remainder!
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees detection of all single-bit errors, double-bit errors, and odd-bit error bursts.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What is the purpose of the ARP (Address Resolution Protocol)?

Options:

- A) Resolves domain names to IP addresses
- B) Maps a known 32-bit IPv4 address to a 48-bit physical MAC address on a local network
- C) Assigns dynamic IP addresses
- D) Encrypts passwords

Answer: **B) Maps a known 32-bit IPv4 address to a 48-bit physical MAC address on a local network** **Detailed Solution & Reasoning:** ARP broadcasts 'Who has IP X.X.X.X? Tell MAC Y.Y.Y.Y' to dynamically resolve physical hardware addresses.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Amazon)

- **Interview Question:** _Why is the minimum frame size in standard Ethernet strictly 64 bytes?_
- **Model High-Scoring Answer:** Ethernet relies on CSMA/CD (Carrier Sense Multiple Access with Collision Detection). For collision detection to succeed, a sending station must still be transmitting bits when a collision reflection from the farthest cable boundary travels back to it ($T_{\text{frame}} \ge 2 \times T_{\text{prop}}$). With a maximum network diameter of 2.5 km and 10 Mbps speed, $2 \times T_{\text{prop}}$ is approx 51.2 microseconds, requiring $512 \text{ bits} = 64 \text{ bytes}$. If a frame were shorter, a collision could occur after transmission completes, preventing the sender from ever detecting that its packet collided.
- **Interviewer Follow-Up & Pro Tip:** ? Cite $T_{\text{transmission}} \ge 2 \times T_{\text{propagation}}$ as the mathematical requirement.

CHAPTER 3

Network Layer: IPv4, IPv6, Subnetting & CIDR

Key Principle / Exam Takeaway: Master IPv4 header fields, Classless Inter-Domain Routing (CIDR), VLSM subnetting calculations, and IPv6 transitions.

Core Curriculum Topics Covered in this Chapter:

- IPv4 Header: Version, IHL, Type of Service, Total Length, TTL, Protocol, Header Checksum
- Classful addressing (Classes A, B, C, D, E) and private IP ranges (RFC 1918)
- Classless Inter-Domain Routing (CIDR) and Variable Length Subnet Masking (VLSM)
- IPv4 fragmentation: Identification, Flags (DF, MF), and Fragment Offset (units of 8 bytes)
- IPv6: 128-bit addresses, fixed 40-byte header, and elimination of checksum/broadcasts

· 1. Core Concepts & In-Depth Explanation

The Network Layer delivers packets from source host to destination host across interconnected networks.

IPv4 Addressing & CIDR: An IPv4 address is 32 bits long (4 bytes), formatted in dotted-decimal notation. Classful addressing wasted vast address blocks and was replaced by **CIDR (Classless Inter-Domain Routing)**:

- Subnet notation: `IP/N`, where N represents the number of contiguous network bits in the subnet mask.
- Available host addresses in a `/N` subnet $= 2^{32 - N} - 2$ (subtracting Network Address and Broadcast Address).

Private IP Address Ranges (RFC 1918):

- Class A: `10.0.0.0/8`
- Class B: `172.16.0.0/12`
- Class C: `192.168.0.0/16`

IPv6: Expands address space to 128 bits (3.4×10^{38} addresses), eliminating the need for NAT, simplifying router processing via a streamlined fixed 40-byte base header, and replacing broadcast with multicast.

· 2. Key Formulas, Rules & Complexity Classes

1. **Host Capacity in CIDR Subnet /N:** $$\text{Total Hosts} = 2^{32 - N} - 2$$ 2. **Fragment Offset Calculation:** $$\text{Fragment Offset} = \frac{\text{Data Byte Offset in Original Packet}}{8}$$

· 3. Step-by-Step Solved Examples

Example 3.1: VLSM Subnet Calculation for /26 Network

- **Difficulty / Target Companies:** `Easy | Amazon, Cisco, TCS`

- **Problem Statement:** Given IP address `192.168.10.130/26`, calculate: Subnet Mask, Network Address, Broadcast Address, and Valid Host Range.

Step-by-Step Walkthrough & Logic:

- Step 1: Prefix `/26` means 26 bits are 1s: $11111111.11111111.11111111.11000000 = 255.255.255.192$.
- Step 2: Host bits $= 32 - 26 = 6$ bits. Block size $= 2^6 = 64$.
- Step 3: Subnet intervals: 0, 64, 128, 192. 130 falls in the third block (128).
- Step 4: Network Address $= 192.168.10.128$.
- Step 5: Broadcast Address $= 128 + 64 - 1 = 192.168.10.191$.
- Step 6: Valid Host Range $= 192.168.10.129$ to $192.168.10.190$ (62 usable hosts).

Complete Solved Code / Calculation:

```
// Subnet Summary for 192.168.10.130/26:
// Netmask: 255.255.255.192
// Network IP: 192.168.10.128
// First Host: 192.168.10.129
// Last Host:  192.168.10.190
// Broadcast:  192.168.10.191
```

- **Line-by-Line Breakdown & Complexity Analysis:** Standard enterprise subnetting formula determining broadcast domains and host capacity.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: How many usable host IP addresses are available in a /28 subnet?

Options:

- A) 16
- B) 14
- C) 30
- D) 6

Answer: **B) 14** **Detailed Solution & Reasoning:** Total addresses = $2^{32 - 28} = 2^4 = 16$. Usable hosts = $16 - 2 = 14$ (excluding network and broadcast addresses).

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Google)

- **Interview Question:** _What is the purpose of the Time-To-Live (TTL) field in the IPv4 header?_
- **Model High-Scoring Answer:** TTL prevents packets from circulating indefinitely in routing loops. The sender initializes TTL (e.g., 64 or 128). Each router that forwards the packet decrements the TTL field by 1. If TTL reaches 0 before the packet reaches its destination, the router drops the packet and transmits an ICMP Time Exceeded (Type 11) message back to the sender. This exact mechanism is leveraged by `traceroute` to map network topology.

- **Interviewer Follow-Up & Pro Tip:** ? Connect TTL expiration to how Traceroute identifies intermediate routers.

CHAPTER 4

Routing Algorithms & Protocols: OSPF, BGP & Distance Vector

Key Principle / Exam Takeaway: Master Intra-domain vs Inter-domain routing, Distance Vector (Bellman-Ford) Count-to-Infinity, Link State (Dijkstra) OSPF, and Path Vector BGP.

Core Curriculum Topics Covered in this Chapter:

- Routing vs Forwarding: Control Plane vs Data Plane
- Distance Vector Routing: Bellman-Ford algorithm, Count-to-Infinity problem, Split Horizon with Poison Reverse
- Link State Routing: Dijkstra's algorithm, Link State Advertisements (LSA), OSPF areas
- Autonomous Systems (AS) and Path Vector Routing: Border Gateway Protocol (BGP)
- Hierarchical routing, software-defined networking (SDN) control separation

· 1. Core Concepts & In-Depth Explanation

Routing determines the global end-to-end path packets travel from source to destination across multiple routers:

- **Distance Vector (RIP):** Routers periodically share their entire routing table with immediate neighbors. Uses Bellman-Ford. Suffers from slow convergence and the **Count-to-Infinity** problem when a link breaks (mitigated via Split Horizon and Poison Reverse).
- **Link State (OSPF):** Routers flood the state of their local links (LSA) across the entire autonomous system. Every router constructs an identical global network topology graph and independently runs Dijkstra's Shortest Path algorithm to build its routing table. Converges rapidly with zero loops.
- **Path Vector (BGP):** The routing protocol of the global Internet. Interconnects Autonomous Systems (AS) by transmitting complete AS path vectors, eliminating routing loops at internet scale based on commercial peering policies.

· 2. Key Formulas, Rules & Complexity Classes

1. **Bellman-Ford Distance Vector Update:** $$D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Tracing Dijkstra's Algorithm in an OSPF Link State Router

- **Difficulty / Target Companies:** `Medium | Cisco, Amazon, Microsoft`
- **Problem Statement:** Given network graph with 4 routers (A, B, C, D) and edge costs A-B: 2, A-C: 5, B-C: 1, B-D: 4, C-D: 2. Compute shortest paths from source A.

Step-by-Step Walkthrough & Logic:

- Step 1: Initialize distances: A:0, B:2, C:5, D:inf. Visited: {A}.
- Step 2: Min unvisited is B (cost 2). Relax neighbors: C becomes min(5, 2 + 1) = 3 (via B); D becomes min(inf, 2 + 4) = 6 (via B). Visited: {A, B}.

- Step 3: Min unvisited is C (cost 3). Relax neighbors: D becomes min(6, 3 + 2) = 5 (via C). Visited: {A, B, C}.
- Step 4: Min unvisited is D (cost 5). Visited: {A, B, C, D}.
- Step 5: Final Shortest Paths from A: to B = 2, to C = 3, to D = 5.

Complete Solved Code / Calculation:

```
// Shortest Path Tree from A:
// Path to B: A -> B (Cost 2)
// Path to C: A -> B -> C (Cost 3)
// Path to D: A -> B -> C -> D (Cost 5)
```

- **Line-by-Line Breakdown & Complexity Analysis:** OSPF builds this identical shortest path tree on all nodes within the area, guaranteeing loop-free shortest paths.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: What mechanism does Distance Vector routing use to prevent immediate two-node routing loops?

Options:

- A) CRC Checksum
- B) Split Horizon with Poison Reverse
- C) Dijkstra's algorithm
- D) Token Ring

Answer: **B) Split Horizon with Poison Reverse** **Detailed Solution & Reasoning:** Split Horizon states that if router A learned a route to destination X through router B, router A must not advertise that route back to router B.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Amazon)

- **Interview Question:** _What is Border Gateway Protocol (BGP) and why does the Internet rely on Path Vector rather than Link State routing between Autonomous Systems?_
- **Model High-Scoring Answer:** BGP is the routing protocol that connects distinct Autonomous Systems (AS) operated by ISPs, universities, and tech giants. Link State algorithms require all routers to maintain an identical global map of every router and link, which would require gigabytes of memory and crash under internet scale. Furthermore, global routing is governed by corporate peering policies (e.g., 'do not transit competitor traffic') rather than shortest mathematical hop counts. BGP exchanges AS Path vectors (`AS 15169 -> AS 3356 -> Destination`), allowing each AS to enforce custom routing policies while preventing loops.
- **Interviewer Follow-Up & Pro Tip:** ? Cite scalability and policy-based routing.

CHAPTER 5

Transport Layer: TCP vs UDP In-Depth

Key Principle / Exam Takeaway: Master TCP 3-Way Handshake, 4-Way Teardown, TIME_WAIT state, Flow Control (Sliding Window), and Congestion Control (Slow Start, AIMD).

Core Curriculum Topics Covered in this Chapter:

- Transport Layer duties: Process-to-process communication via Port numbers
- TCP vs UDP: Reliability, ordering, connection state, and overhead comparisons
- TCP 3-Way Handshake (SYN, SYN-ACK, ACK) and 4-Way Connection Teardown (FIN, ACK)
- TCP TIME_WAIT state and 2MSL (Maximum Segment Lifetime) rationale
- TCP Congestion Control: Slow Start, Congestion Avoidance, Fast Retransmit, Fast Recovery (AIMD)

· 1. Core Concepts & In-Depth Explanation

The Transport Layer provides logical end-to-end communication between application processes:

- **UDP (User Datagram Protocol):** Lightweight, connectionless, unreliable transport with an 8-byte header. Zero handshake latency, no flow or congestion control. Ideal for real-time video streaming, VoIP, DNS queries, and gaming.
- **TCP (Transmission Control Protocol):** Connection-oriented, full-duplex, reliable byte stream with a minimum 20-byte header. Guarantees in-order delivery via sequence numbers and ACKs, retransmits lost packets, and matches sender velocity to network capacity.

TCP 3-Way Handshake: 1. Client sends `SYN` (Sequence Number = x). 2. Server responds with `SYN-ACK` (Sequence Number = y, Acknowledgment Number = $x + 1$). 3. Client replies with `ACK` (Acknowledgment Number = $y + 1$).

Congestion Control (AIMD): Controls traffic based on network bottleneck capacity:

- **Slow Start:** Exponential growth of Congestion Window (`cwnd`) doubling every RTT until reaching `ssthresh`.
- **Congestion Avoidance:** Linear growth ($+1 \text{ MSS}$ per RTT).
- **On Packet Loss:** Additive Increase, Multiplicative Decrease (halves `cwnd`).

· 2. Key Formulas, Rules & Complexity Classes

1. **AIMD Invariant:** $$\text{On success: } cwnd = cwnd + 1 \quad \text{(Linear additive increase)}$$ $$\text{On loss: } cwnd = \frac{cwnd}{2} \quad \text{(Multiplicative decrease)}$$ 2. **TCP TIME_WAIT Duration:** $$\text{TIME_WAIT} = 2 \times \text{MSL (Maximum Segment Lifetime, default 120s)}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Tracing Congestion Window (cwnd) Evolution across RTTs

- **Difficulty / Target Companies:** `Medium | Google, Amazon, Cisco`

- **Problem Statement:** TCP starts in Slow Start with $cwnd = 1$ MSS and $ssthresh = 8$ MSS. Track $cwnd$ over 6 successful RTTs.

Step-by-Step Walkthrough & Logic:

- RTT 1: cwnd = 1 (Slow Start, doubles).
- RTT 2: cwnd = 2.
- RTT 3: cwnd = 4.
- RTT 4: cwnd = 8 (Reaches ssthresh!).
- RTT 5: Enters Congestion Avoidance (linear growth): cwnd = 8 + 1 = 9.
- RTT 6: cwnd = 9 + 1 = 10.

Complete Solved Code / Calculation:

```
// cwnd sequence across RTTs:
// RTT 1: 1 MSS  (Slow Start)
// RTT 2: 2 MSS
// RTT 3: 4 MSS
// RTT 4: 8 MSS  (Hits threshold)
// RTT 5: 9 MSS  (Congestion Avoidance - linear)
// RTT 6: 10 MSS
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates the transition from exponential growth to linear conservation.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: Why does the initiating side of a TCP connection enter the TIME_WAIT state after sending the final ACK during connection teardown?

Options:

- A) To download remaining files
- B) To ensure the final ACK is delivered and allow duplicate in-flight packets to expire in the network before port reuse
- C) Because the OS is rebooting
- D) To save power

Answer: **B) To ensure the final ACK is delivered and allow duplicate in-flight packets to expire in the network before port reuse**
Detailed Solution & Reasoning: If the final ACK is lost, the server retransmits its FIN. TIME_WAIT ensures the client can retransmit the final ACK and ensures stale packets disappear.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _What is the difference between Flow Control and Congestion Control in TCP?_

- **Model High-Scoring Answer:** Flow Control is an end-to-end mechanism that prevents a fast sender from overwhelming a slow *receiver*. The receiver advertises its available buffer capacity in the TCP `Window Size` field (`rwnd`). The sender never transmits more bytes than `rwnd`. Congestion Control prevents senders from overwhelming the intermediate *network routers and links*. The sender maintains an internal Congestion Window (`cwnd`) inferred through latency and packet drops. The effective transmission window is $\min(rwnd, cwnd)$.

- **Interviewer Follow-Up & Pro Tip:** ? Highlight: Flow Control protects the receiver; Congestion Control protects the network.

CHAPTER 6

Application Layer Protocols: HTTP, DNS, DHCP & TLS

Key Principle / Exam Takeaway: Master HTTP/1.1 vs HTTP/2 vs HTTP/3 (QUIC), DNS recursive resolution, DHCP DORA process, and TLS 1.3 handshake.

Core Curriculum Topics Covered in this Chapter:

- DNS (Domain Name System): Root, TLD, Authoritative nameservers, Recursive vs Iterative queries
- DHCP: Discover, Offer, Request, Acknowledge (DORA) process
- HTTP Evolution: HTTP/1.1 (Keep-Alive, Head-of-Line blocking), HTTP/2 (Multiplexing, binary framing, HPACK), HTTP/3 (QUIC over UDP)
- TLS 1.3 Cryptographic Handshake, Public Key Infrastructure, Certificates
- REST vs GraphQL vs gRPC RPC paradigms

· 1. Core Concepts & In-Depth Explanation

The Application Layer contains protocols user programs interact with directly:

- **DNS (Port 53 UDP/TCP):** Translates human domain names (`google.com`) into IP addresses. Traverses Local Resolver -> Root Nameserver (`.`) -> TLD Nameserver (`.com`) -> Authoritative Nameserver.
- **DHCP (Ports 67/68 UDP):** Dynamic host configuration using **DORA**: Discover (broadcast) -> Offer (unicast/broadcast) -> Request (broadcast) -> Acknowledge (unicast).
- **HTTP/1.1 vs HTTP/2 vs HTTP/3:**
- **HTTP/1.1:** Text-based protocol. Suffers from **Head-of-Line (HoL) Blocking** where slow responses block subsequent requests on the same TCP connection.
- **HTTP/2:** Binary framing protocol. Enables request multiplexing (multiple bidirectional streams over a single TCP connection) and HPACK header compression. Still suffers from TCP-level HoL blocking on packet loss.
- **HTTP/3:** Replaces TCP with **QUIC** running over UDP. Delivers true zero-RTT connection establishment and independent packet streams: packet loss on one stream does not pause other streams!

· 2. Key Formulas, Rules & Complexity Classes

1. **HTTP Generation Progression:** $$\text{HTTP/1.1 (TCP, text, HoL blocking)} \to \text{HTTP/2 (TCP, binary, multiplexed)} \to \text{HTTP/3 (QUIC/UDP, 0-RTT)}$$

· 3. Step-by-Step Solved Examples

Example 6.1: Tracing Complete What Happens When You Type google.com in a Browser

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Microsoft`
- **Problem Statement:** Detail the full network lifecycle from typing google.com until the webpage renders.

Step-by-Step Walkthrough & Logic:

- Step 1: Browser parses URL, checks browser cache, OS hosts file, and DNS cache.

- Step 2: DNS Resolution: Local resolver queries Root, TLD (.com), and Authoritative nameservers to resolve IP.
- Step 3: TCP Connection: Client executes 3-Way Handshake (SYN, SYN-ACK, ACK) with web server on port 443.
- Step 4: TLS 1.3 Handshake: Client and server exchange keys via Diffie-Hellman, server presents X.509 certificate, session keys established.
- Step 5: HTTP Request: Browser transmits `GET / HTTP/2` request.
- Step 6: Server processes request, streams HTML response.
- Step 7: Browser parses HTML, builds DOM/CSSOM, requests referenced assets, and renders page.

Complete Solved Code / Calculation:

```
// End-to-end Lifecycle:
// DNS -> TCP Handshake -> TLS Handshake -> HTTP Request -> Render Engine
```

- **Line-by-Line Breakdown & Complexity Analysis:** Flagship interview scenario testing full-stack network and systems fluency.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What is the primary innovation of HTTP/3 over HTTP/2?

Options:

- A) Larger cookies
- B) Uses QUIC protocol over UDP to eliminate transport-layer Head-of-Line blocking
- C) Replaces JSON with XML
- D) Uses 64-bit addresses

Answer: **B) Uses QUIC protocol over UDP to eliminate transport-layer Head-of-Line blocking** **Detailed Solution & Reasoning:** By replacing TCP with QUIC over UDP, individual lost packets only pause their specific stream rather than stalling all multiplexed streams.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Google)

- **Interview Question:** _How does DNS caching work and what role does TTL play?_
- **Model High-Scoring Answer:** Every DNS resource record (A, CNAME, MX) includes a Time-To-Live (TTL) integer specifying the number of seconds the record can be cached. Operating systems, local resolvers, and browser engines store records locally. When a domain is requested again before TTL expires, the IP is returned immediately from local RAM cache without querying root or authoritative nameservers. Once TTL expires, the cache purges the record and executes a fresh authoritative lookup.
- **Interviewer Follow-Up & Pro Tip:** ? Cite lower TTLs during migrations and higher TTLs for performance.

CHAPTER 7

Network Security, Firewalls & Cryptography

Key Principle / Exam Takeaway: Master Symmetric vs Asymmetric encryption, RSA, Diffie-Hellman, digital signatures, firewalls, and DDoS mitigation.

Core Curriculum Topics Covered in this Chapter:

- CIA Triad: Confidentiality, Integrity, Availability
- Symmetric encryption (AES) vs Asymmetric encryption (RSA, ECC)
- Key Exchange: Diffie-Hellman key exchange and Perfect Forward Secrecy (PFS)
- Hashing and Integrity: SHA-256, HMAC, and digital signatures
- Firewalls: Packet filtering (Stateless) vs Stateful Inspection vs Next-Gen Application Firewalls

· 1. Core Concepts & In-Depth Explanation

Network Security secures data confidentiality, prevents tampering, and ensures system availability:

- **Symmetric Encryption (AES):** Uses a single shared secret key for encryption and decryption. Extremely fast computationally.
- **Asymmetric Encryption (RSA, ECC):** Uses a mathematically linked key pair: Public Key (encrypts data or verifies signatures) and Private Key (decrypts data or creates signatures). Computationally heavy.
- **Hybrid Cryptography (TLS):** Uses Asymmetric cryptography (Diffie-Hellman) only during the handshake to authenticate identity and securely negotiate a shared ephemeral session key. Once established, all bulk payload data is encrypted using high-speed Symmetric AES.
- **Stateful Firewalls:** Track the state of active network connections (SYN sent, established) in a state table, automatically allowing return traffic matching outbound requests while blocking unsolicited inbound connections.

· 2. Key Formulas, Rules & Complexity Classes

1. **Diffie-Hellman Key Exchange:** $$A = g^a \pmod p, \quad B = g^b \pmod p$$ $$\text{Shared Secret: } K = B^a \pmod p = A^b \pmod p = g^{ab} \pmod p$$

· 3. Step-by-Step Solved Examples

Example 7.1: Verifying Digital Signatures for Software Authenticity

- **Difficulty / Target Companies:** `Easy | Amazon, Microsoft`
- **Problem Statement:** Explain how a developer signs a software update and how an end-user client verifies it hasn't been tampered with.

Step-by-Step Walkthrough & Logic:

- Step 1: Developer hashes the software executable using SHA-256.
- Step 2: Developer encrypts the hash using their **Private Key**; this encrypted hash is the **Digital Signature**.
- Step 3: User receives software binary and signature.

- Step 4: User hashes received binary using SHA-256 -> Hash A.
- Step 5: User decrypts signature using Developer's public key -> Hash B.
- Step 6: If Hash A == Hash B, software is authentic and unaltered!

Complete Solved Code / Calculation:

```
// Digital Signature Contract:
// Sign:   Signature = Encrypt(Hash(Data), PrivateKey)
// Verify: Hash(Data) == Decrypt(Signature, PublicKey)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees both Authentication (only private key holder could sign) and Integrity (any file modification changes the hash).

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: What is Perfect Forward Secrecy (PFS) in TLS?

Options:

- A) Faster encryption speed
- B) Compromise of a server's long-term private key does not compromise past session keys because session keys are ephemeral
- C) Zero packet drops
- D) Prevents phishing

Answer: **B) Compromise of a server's long-term private key does not compromise past session keys because session keys are ephemeral** **Detailed Solution & Reasoning:** Ephemeral Diffie-Hellman generates temporary keys per session; recording past encrypted traffic does not enable retroactive decryption.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Amazon)

- **Interview Question:** _What is a SYN Flood attack and how do SYN Cookies defend against it?_
- **Model High-Scoring Answer:** In a SYN Flood attack, an attacker sends thousands of TCP SYN packets with spoofed source IPs and never responds to the server's SYN-ACKs. The server allocates a Transmission Control Block (TCB) in its SYN queue for each half-open connection, rapidly exhausting memory and rejecting legitimate users. **SYN Cookies** eliminate the need to allocate memory for half-open connections: the server encodes the client state, timestamps, and MSS into the initial sequence number of the SYN-ACK using a cryptographic hash. The server allocates memory only when the legitimate client returns the final ACK with the echoed sequence number.
- **Interviewer Follow-Up & Pro Tip:** ? Clearly describe stateless half-open connection management.

CHAPTER 8

Solved Placement Networking Scenarios & System Diagnostics

Key Principle / Exam Takeaway: Master Linux networking utilities (ping, traceroute, netstat, ss, tcpdump), NAT traversal, socket programming, and troubleshooting scenarios.

Core Curriculum Topics Covered in this Chapter:

- Network diagnostics: ping (ICMP Echo), traceroute (TTL expiry), netstat/ss, nslookup/dig
- Packet analysis with Wireshark and tcpdump command-line capture
- Network Address Translation (NAT): SNAT, DNAT, Port Address Translation (PAT)
- Socket programming fundamentals in Python/C: socket(), bind(), listen(), accept(), connect()
- Comprehensive campus placement networking interview rubrics

· 1. Core Concepts & In-Depth Explanation

This capstone chapter focuses on real-world systems debugging, packet capture inspection, and production network diagnostics:

- **`ping`:** Sends ICMP Echo Request (Type 8) and listens for ICMP Echo Reply (Type 0). Tests end-to-end host reachability and round-trip latency.
- **`traceroute`:** Sends packets with increasing TTL values ($1, 2, 3 \dots$). When packet hits router with TTL 1, it drops and returns ICMP Time Exceeded, revealing the intermediate router's IP.
- **Network Address Translation (NAT/PAT):** Maps multiple private IP addresses behind a single public IP by tracking unique dynamic source port numbers in a NAT translation table.
- **Socket Programming:** The operating system API for network communications. A server binds to a port and listens; client initiates connection; kernel creates a dedicated connected socket descriptor for data transfer.

· 2. Key Formulas, Rules & Complexity Classes

1. **Socket System Call Sequence:** $$\text{Server: } socket() \to bind() \to listen() \to accept() \to read()/write() \to close()$$ $$\text{Client: } socket() \to connect() \to write()/read() \to close()$$

· 3. Step-by-Step Solved Examples

Example 8.1: Minimal TCP Echo Server in Python Socket API

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** Write a complete, working TCP Echo server listening on port 8080 that reads messages and echoes them back.

Step-by-Step Walkthrough & Logic:

- Step 1: Create TCP socket with `AF_INET` and `SOCK_STREAM`.
- Step 2: Bind to localhost on port 8080 and listen for connections.
- Step 3: Accept client connection, receive data in a loop, echo back, and close.

Complete Solved Code / Calculation:

```
import socket

def run_echo_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 8080))
    server.listen(5)
    print("Echo server listening on port 8080...")

    while True:
        client_sock, client_addr = server.accept()
        print(f"Connection from {client_addr}")
        data = client_sock.recv(1024)
        if data:
            client_sock.sendall(data) // Echo back received bytes
        client_sock.close()
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates the fundamental socket lifecycle implemented across all web servers.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: Which diagnostic command displays all active listening TCP ports and their associated process IDs in modern Linux?

Options:

- A) ping -a
- B) ss -tulpn
- C) ifconfig -s
- D) traceroute -p

Answer: **B) ss -tulpn** **Detailed Solution & Reasoning:** ss (Socket Statistics) is the high-performance modern replacement for netstat.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Google)

- **Interview Question:** _You can ping a server by its IP address (e.g. 8.8.8.8), but you cannot load google.com in your browser. What is the root cause?_
- **Model High-Scoring Answer:** Since pinging raw IP addresses succeeds, the physical network, Data Link frames, and Layer 3 IP routing are fully functional. The failure to access domain names points directly to a DNS configuration issue. The machine's local DNS resolver (`/etc/resolv.conf` on Linux or Network Adapter DNS settings on Windows) is either misconfigured, pointing to an unreachable DNS server, or port 53 UDP traffic is blocked by a firewall.

- **Interviewer Follow-Up & Pro Tip:** ? Isolate the network layer from the application DNS layer.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.